ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Data Analytics
Data Normalization
Scaling

Faculty of Information Technology

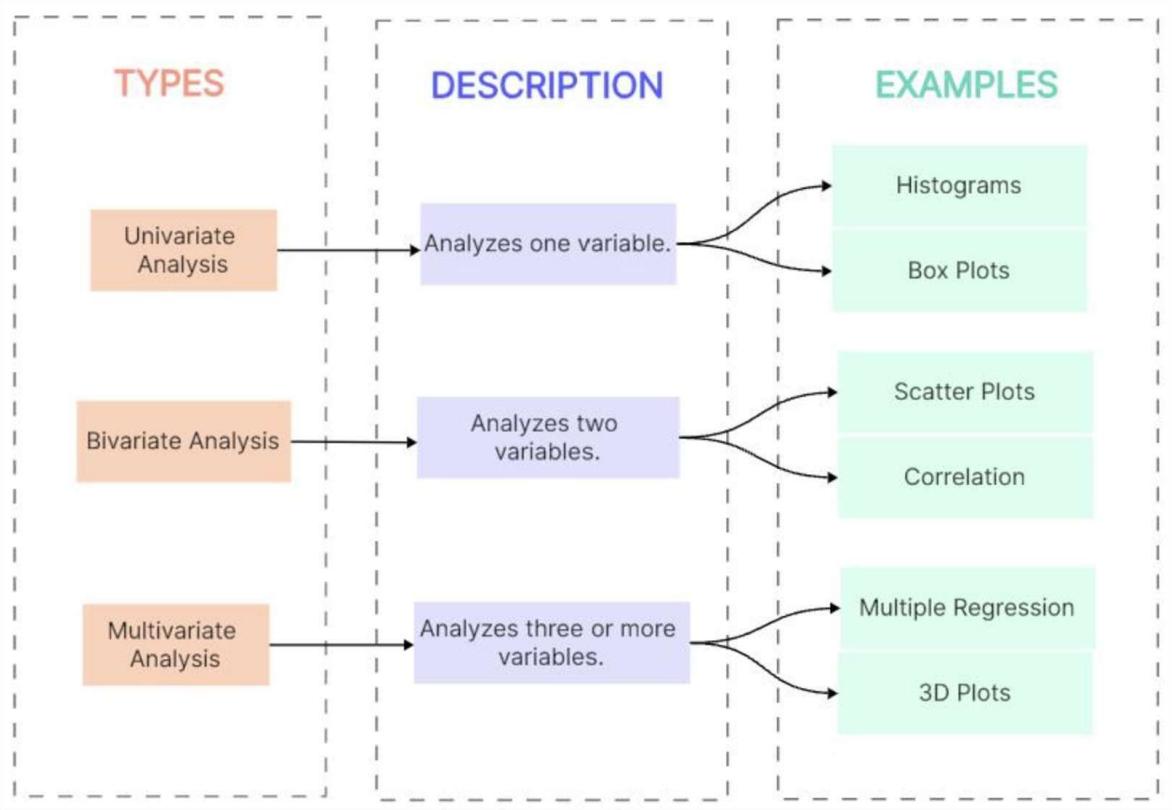
ĐẠI HỌC HOA SEN

Objectives

- ✓ Data Analysis
- ✓ Steps
- ✓ Missing Data - Data imputation
- ✓ Data Normalization

Data Analysis

- **Data analysis** is the systematic process of examining datasets to uncover patterns, trends, and relationships between variables.



- **Univariate Analysis**: Focuses on describing a single characteristic or data point in isolation (e.g., the height of a group of people).
  - Histograms, Box Plots
- **Bivariate Analysis**: Explores the relationship or connection between two different variables (e.g., how height relates to weight).
  - Scatter Plots, Correlation
- **Multivariate Analysis**: Examines complex interactions between multiple variables simultaneously to understand how they influence one another (e.g., how height, weight, and age all relate to heart rate).
  - Multiple Regression, 3D Plots

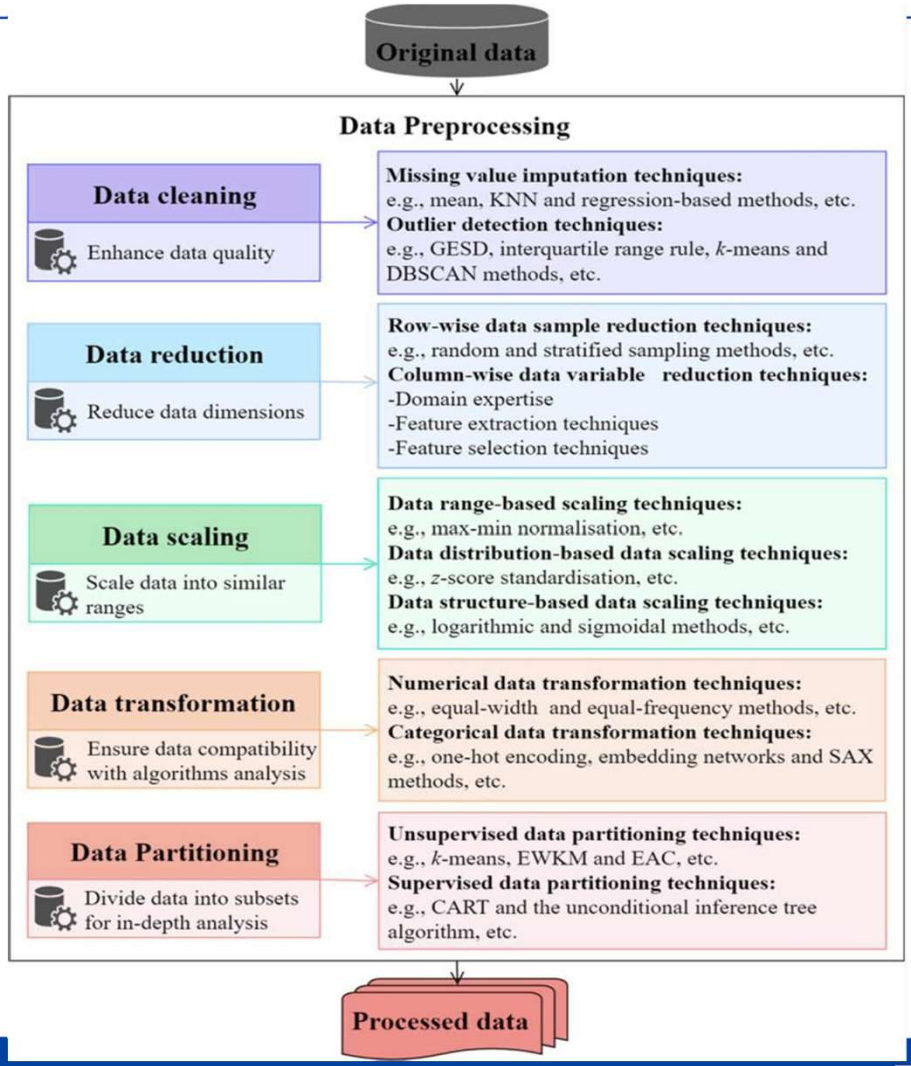
Steps of data analytics

1. **Data Requirement:** This is the planning phase where you define **why** you are doing the analysis. You identify the specific questions you need to answer and what metrics will define success.
2. **Data Collection:** Once you know what you need, you gather the raw information. This can come from various sources like databases, surveys, web scraping, or existing logs.
3. **Data Processing:** Raw data is often disorganized. In this step, the data is structured into a usable format (like a spreadsheet or database table) so it can be effectively handled by analysis tools.

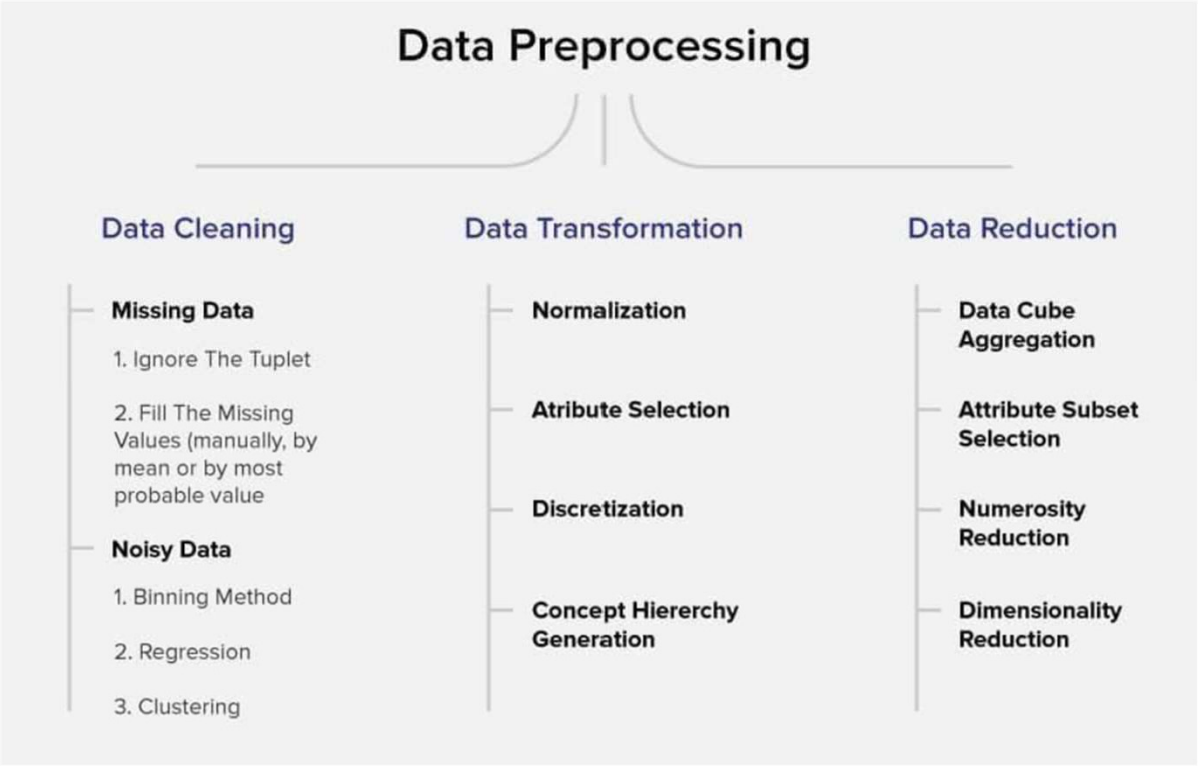
Steps of data analytics (cont.)

4. **Data Cleaning:** This is often the most time-consuming step. You identify and fix errors, remove duplicates, and handle missing information to ensure the data is accurate and reliable.
5. **Data Analysis:** Now the actual "work" begins. Using statistical tools or software, you look for patterns, trends, and correlations that answer the questions you set in step one.
6. **Data Interpretation:** Finally, you translate the findings into a story. This involves creating visualizations (like charts) and summarizing what the results actually mean for the business or project.

Data Processing



Data Processing



Missing Data

Missing values can be broadly classified into the following 3 types:

1. **Missing Completely at Random (MCAR) —** this type of missing value does not have any relation with any other value in the dataset. There is no pattern or dependency on other values. This could be due to some human error, error in data loading, failure of system, etc.
2. **Missing at Random (MAR) —** This is the type of missing value that has some sort of relationship or dependency on other values in the dataset but not with the missing data. A pattern can be observed in the missing data and hence, the probability of missing values is dependent on other values. For example, a dataset having a column 'drinks alcohol' might have missing values for people having less age, as people below the age of 21 might not be comfortable sharing this information.
3. **Missing Not at Random (MNAR) —** The type of data is missing not at random (MNAR) if the pattern in the missing values is not dependent on the other values in the dataset. For example, it might be because some people are unwilling to answer some questions in a survey. Like, people who are overweight might not be willing to share their weight.

Data imputation

- Data imputation is the process of replacing missing values in a dataset with estimated values.
- a statistical technique used to handle incomplete datasets
- allows for more complete analysis and prevents the loss of data points that would occur if rows with missing values were simply deleted.
- deleting rows with missing data can introduce bias and reduce the efficiency of the analysis, so imputation is preferred as it preserves all cases.

How to deal with missing data?

- **Deletion** (Removal):
  - Listwise Deletion (Complete Case Analysis): Removing the entire row if any value is missing. Suitable if missingness is rare and completely at random.
  - Dropping Columns: If a column has too many missing values (e.g., >70%), it may be best to remove it.
- **Imputation** (Filling in Value):
  - Mean/Median/Mode Imputation: Replacing missing numerical values with the mean or median of the feature, or categorical values with the mode. Median is better for skewed data.
  - Forward/Backward Fill: Used in time-series data to propagate the last valid observation forward or next valid backward.
  - Regression Imputation: Using other variables in the dataset to predict and fill in the missing value.
  - K-Nearest Neighbors (KNN): Filling values based on the most similar data points.

MICE

- Multiple Imputation by Chained Equations (MICE)
  - An advanced and currently the most effective statistical method for handling missing data by replacing missing values with multiple different predicted values (multiple imputation), based on the relationships between variables in the dataset.
  - Operates through an iterative mechanism, building regression models (chained equations) for each variable with missing data, using the remaining variables as predictors, creating multiple complete datasets to minimize errors.

MICE Algorithm

Step 1: Initialize Missing Values

- First, all missing values are temporarily replaced using simple methods such as:
- Mean
- Median
- Mode
- Random sampling
- This creates an initial complete dataset.

Step 2: Select One Variable with Missing Values

- Suppose the variable with missing data is:
- X_1
- Temporarily treat:
  - X_1 as the target variable
  - $X_2, X_3, \ldots, X_n$ as predictor variables

Step 3: Build a Prediction Model

- Use the observed values of X_1 to train a predictive model such as:
- Linear Regression → for numerical data
- Logistic Regression → for binary data
- Multinomial Regression → for categorical data
- Predictive Mean Matching (PMM)
- Random Forest (advanced version)
- The model can be written as:
  - $X_1 = f(X_2, X_3, .., X_n)$

Step 4: Impute Missing Values

- Use the trained model to predict and replace the missing values of X_1.

Step 5: Repeat for Other Variables

- Continue the same process for:
  - $X_2, X_3, \ldots, X_n$
- Each variable is treated as the target variable one at a time.

- **Step 6: Complete One Iteration**
  - After all variables with missing values have been imputed once, one full iteration is completed.

Step 7: Repeat Until Convergence

- Repeat the iterations multiple times until the imputed values become stable.
- Usually:
  - 5–10 iterations
  - or 20+ for complex datasets
  - This is called **convergence**.

Step 8: Create Multiple Imputed Datasets

- Instead of generating only one dataset, MICE creates:
  - 5 datasets
  - 10 datasets
  - or more
- This helps reflect the uncertainty of missing values.

Step 9: Analyze and Pool Results

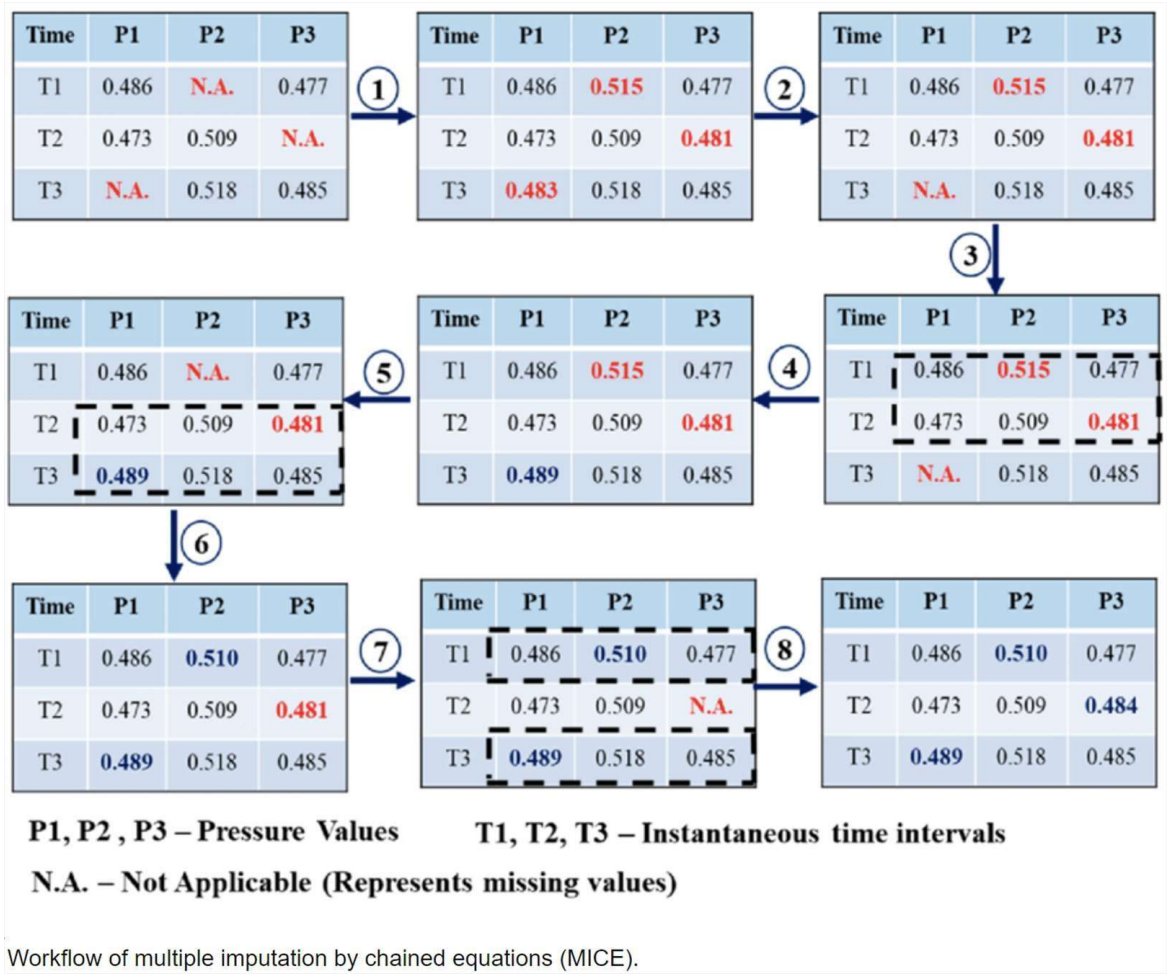
- Perform statistical analysis on each completed dataset separately, then combine the results using:
  - **Rubin's Rules**
  - This provides more reliable final estimates.

- **Rubin's Rules** is a statistical method used to combine (pool) the results from **multiple imputed datasets** created by methods like **MICE (Multiple Imputation by Chained Equations)**.
- Instead of analyzing only one completed dataset, we analyze several imputed datasets separately and then combine the results to obtain one final estimate.
- This helps account for the uncertainty caused by missing data.

Pseudocode

- Initialize missing values
- Repeat until convergence:
  For each variable Xi with missing values:
  Remove current imputations of Xi
  Fit model: Xi ~ other variables
  Predict missing Xi
  Replace missing Xi

  Repeat to create m complete datasets
- Analyze each dataset separately
- Pool results using Rubin's Rules



Workflow of multiple imputation by chained equations (MICE).

Feature selection

- Feature selection (variable selection:
  - Is the process of reducing a dataset's complexity by choosing only the most relevant, non-redundant input features (variables) to train machine learning models.
  - It eliminates noisy, irrelevant data to improve model accuracy, speed up training time, and enhance model interpretability.
  - It is often referred to as variable selection or feature subset selection

Techniques

- **Filter Methods**: Using statistical techniques (e.g., Pearson's correlation, Chi-Squared test) to rank and select features based on their relationship with the target variable.
- **Wrapper Methods**: Using a specific model to evaluate subsets of features, such as [Recursive Feature Elimination (RFE)] to find the best-performing combination.
- **Embedded Methods**: Algorithms that perform feature selection during training, such as [LASSO regression] or [Random Forest feature importance].
- **Reducing High Dimensionality**: Dropping unnecessary features in complex datasets, such as in genomic data analysis or financial forecasting, to prevent overfitting.

Wrapper-Baesd Methods

- The basic idea is:
  - Select a subset of features
  - Train a machine learning model using that subset
  - Evaluate model performance (accuracy, precision, RMSE, etc.)
  - Compare with other subsets
  - Choose the best-performing subset

Common Wrapper Methods

- **1. Forward Selection**
  - Starts with:
    - No features
  - Then:
    - Add one feature at a time
  - Choose the feature that improves model performance the most.
  - Continue until no significant improvement occurs.
- **2. Backward Elimination**
  - Starts with:
    - All features
  - Then:
    - Remove one feature at a time
    - Remove the least useful feature first.
  - Continue until only important features remain.

- **3. Recursive Feature Elimination (RFE)**
  - Repeatedly:
    - Train the model
    - Rank feature importance
    - Remove the least important feature
- Continue until the desired number of features is reached.

Recursive Feature Elimination (RFE)

The process follows these six steps:

- **Start with All Features**: You begin the analysis by including every available feature (variable) from your training dataset.
- **Train a Model**: A machine learning model (e.g., Linear Regression, SVM, or Random Forest) is fitted to the data. This model must be able to assign weights or importance scores to each feature.
- **Rank Features by Importance**: The algorithm evaluates how much each feature contributes to the model's predictions using metrics like coefficients or feature importance scores.
- **Remove Least Important Features**: The feature(s) with the lowest importance ranking are pruned (deleted) from the current set.
- **Repeat the Process**: The model is retrained on the remaining features, and the ranking and removal steps are repeated recursively.
- **Finalize the Selected Features**: The loop continues until a pre-defined number of features is reached or an optimal subset is identified, resulting in a more efficient and interpretable model.

Data Normalization

What is Normalization?

- Normalization is a specific form of feature scaling that transforms the range of features to a standard scale.
- Normalization is required only when your dataset has features of varying ranges
- Data normalization is the process of reorganizing data within a database so that users can utilize it for further queries and analysis.
- Simply put, it is the process of developing clean data. This includes eliminating redundant and unstructured data and making the data appear similar across all records and fields.

- **Why Normalize Data?**
  - Normalized data enhances model performance and improves the accuracy of a model. It aids algorithms that rely on distance metrics, such as **k-nearest neighbors** by preventing features with larger scales from dominating the learning process.
  - Normalization fosters stability in the optimization process, promoting faster convergence during gradient-based training. It mitigates issues related to vanishing or exploding gradients, allowing models to reach optimal solutions more efficiently.
  - Normalized data is also easy to interpret and thus, easier to understand. When all the features of a dataset are on the same scale, it also becomes easier to identify and visualize the relationships between different features and make meaningful comparisons.

- Example
  - Square footage: Ranges from 500 to 5000 square feet
  - Number of bedrooms: Ranges from 1 to 5
  - Distance to supermarket: Ranges from 0.1 to 10 miles

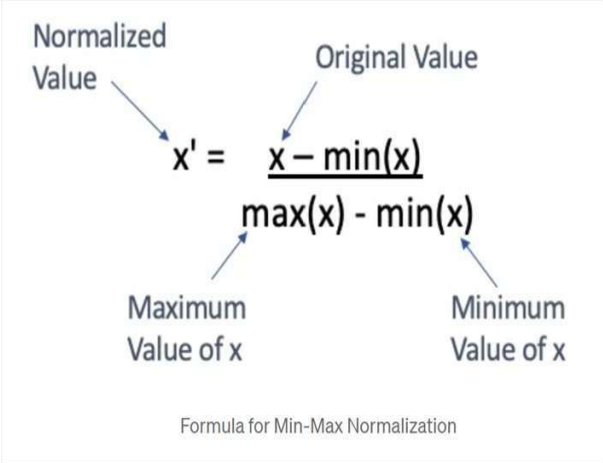
Example (cont.)

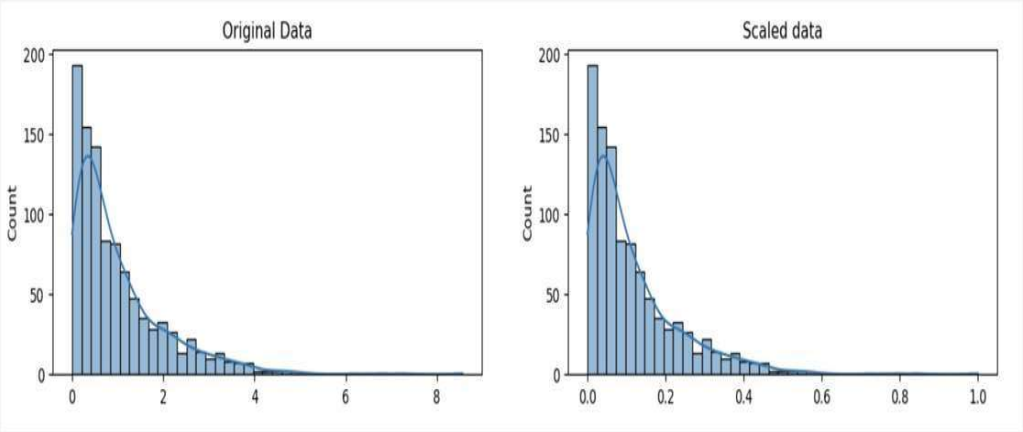
- **Algorithm might give more weight to features with large scales**, such as square footage. During training, the algorithm assumes that a change in total square footage will have a significant impact on housing prices.
- The algorithm might overlook the nuances of features that are relatively small, such as the number of bedrooms and distance to the supermarket. This skewed emphasis can lead to suboptimal model performance and biased predictions.
- By normalizing the features, we can ensure that **each feature contributes proportionally** to the model's learning process. The model can now learn patterns across all features more effectively, leading to a more accurate representation of the underlying relationships in the data.

- Ensure equal feature importance:
  - Normalization scales features so that one feature with a large range (e.g., income) doesn't unfairly dominate other features with smaller ranges (e.g., age) in distance- based calculations.
  - If one feature ranges from 0–1 and another from 0–10,000, many models will treat the larger-scale feature as more important, even if it isn’t. Normalization prevents this.
- Enhance accuracy:
  - For certain models, normalization can lead to more accurate predictions by preventing features with large values from disproportionately influencing the model.
- Improve algorithm performance:
  - To make distance-based algorithms work correctly: Algorithms like k-NN, k-means, and SVM rely on distance. Without normalization, distances become skewed and the model performs poorly.
- Speed up training

Normalization Techniques

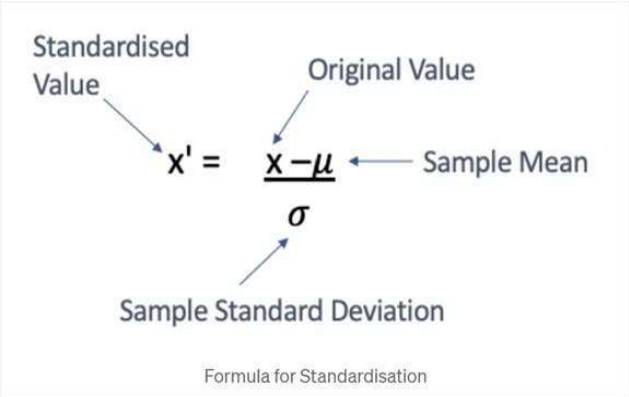
- **Min–Max Normalization** (rescaling):
  - Min-max scaling is very often simply called ‘normalization.’
  - It transforms features to a specified range, typically [0 - 1]





- Min-max scaling is a good choice when:
  - The approximate upper and lower bounds of the dataset is known, and the dataset has few or no outliers
  - When the data distribution is unknown or non-Gaussian, and the data is approximately uniformly distributed across the range
  - When maintaining the distribution's original shape is essential

- **Z-score normalization** (standardization):
  - Assumes a Gaussian (bell curve) distribution of the data and transforms features to have a mean (μ) of 0 and a standard deviation (σ) of 1.



Formula for Standardisation

  - This technique is particularly useful when dealing with algorithms that assume normally distributed data, such as many linear models. (e.g., linear models, logistic regression)

- **Maxabs scaling**

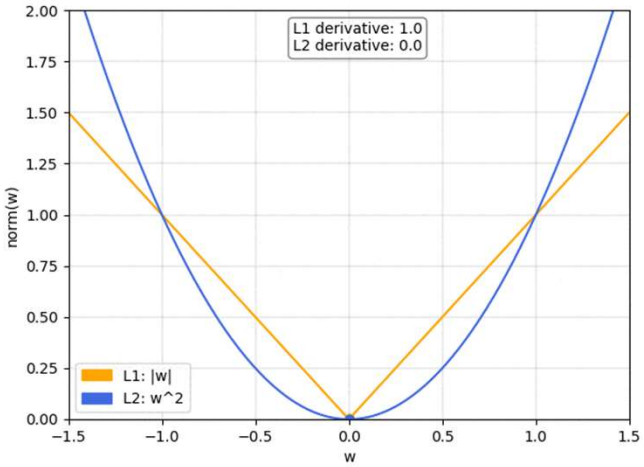
$$X_{\text{scaled}} = \frac{X}{|X_{\max}|}$$

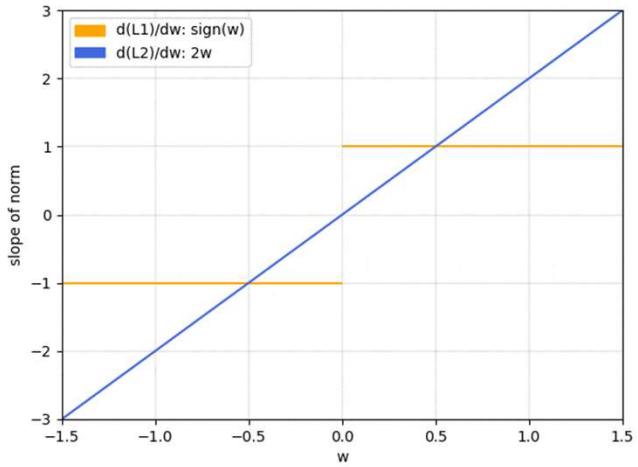
Where:

- X: The original value of the feature.
- $X_{\max}$: The maximum absolute value in the feature.

- Scales to **[-1, 1]** but keeps sparsity (good for sparse matrices)
- Working with sparse data (e.g., text features)

- **L1 or L2 normalization** (vector normalization)
  - L1 normalization (Lasso) uses absolute values to scale data and promotes sparsity, often forcing irrelevant feature weights to zero, making it ideal for feature selection.
  - L2 normalization (Ridge) uses squared values, shrinking weights evenly to reduce the impact of outliers and handling multicollinearity better.





- **One Hot encoding**
  - A technique that converts categorical data into a numerical, binary format (0s and 1s) for machine learning models. It creates new binary columns—often called dummy variables—for each unique category, marking a '1' in the corresponding column and '0' elsewhere.
  - This eliminates false ordinal relationships between categories.

Key Aspects of One-Hot Encoding:

- **Process:** Each unique value in a categorical column is converted into a separate binary feature.
- **Purpose:**
  - Machine learning algorithms require numerical input, and one-hot encoding allows categorical data to be represented in a way they can understand.
- **Example:** A "Color" column with values "Red", "Blue", "Green" becomes three columns: Color_Red, Color_Blue, Color_Green. A row with "Red" becomes [1, 0, 0].

id	color
1	red
2	blue
3	green
4	blue

id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

Key Aspects of One-Hot Encoding (cont.):

- **Handling Multicollinearity:** To avoid the "dummy variable trap" (where columns are perfectly correlated), it is standard practice to drop one of the encoded columns, especially for linear models.
- **Tooling:** Popular libraries for this include *sklearn.preprocessing.OneHotEncoder* from scikit-learn or *pandas.get_dummies* in Python.

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50


Q&A